

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_22

22. State Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the State pattern.

GoF Definition

It allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

Concept

The GoF definition is easy to understand. It states that an object can change what it does and this depends on its current state.

Suppose you are dealing with a large-scale application where the code-base is rapidly growing. As a result, the situation becomes complex and you may need to introduce lots of `if-else` blocks/`switch` statements to guard the various conditions. The State pattern fits in such a context. It allows your objects to behave differently based on their current states and you can define state-specific behaviors with different classes.

In this pattern, start thinking in terms of possible states of your application and segregate the code accordingly. Ideally, each state is independent of the other states. You keep track of these states, and your code responds as per the behavior of the current state. For example, suppose you are watching a program on your television (TV). If you press the Mute button on the TV's control panel, you will notice a state change in your TV. But you won't notice any change if the TV is already in the switched-off mode.

The basic idea is if your code can track the current state of the application, you can centralize the task, segregate your code, and respond ac-

cordingly. This is why in this chapter you'll see state-specific behaviors in separate classes. In short, this is the key concept in the State pattern.

Real-Life Example

The functionalities of a traffic signal or television can be considered as examples of the State pattern. For example, you can change the channel if the TV is already in the switched-on mode. It will not respond to the channel change request if it is in the switched-off mode.

Computer World Example

Here are some examples for you:

- Consider the scenario of a network connection, say a TCP connection. An object can be in various states. For example, a connection might already be established, a connection might be closed, or the object has already started listening through the connection. When this connection receives a request from other objects, it responds according to its present state.
- Consider another example. Suppose you have a job processing system that can process a certain number of jobs at a time. When a new job appears, either the system processes the job or it signals that it is busy with the maximum number of jobs that it can process at that time. This busy signal simply indicates that its total number of job-processing capabilities has been reached and the new job request cannot be fulfilled immediately.

Implementation

This example models the functionalities related to a television, which has a control panel to support on, off, and mute operations. For simplicity, assume that at any given point in time the TV can be in either of these three states: On, Off, or Muted. So, there is an interface called `IPossibleStates` as follows:

```
interface PossibleState {  
    // Users can press any of these buttons-On, Off or Mute  
    void pressOnButton(Television tvContext);  
    void pressOffButton(Television tvContext);  
    void pressMuteButton(Television tvContext);  
}
```

Three concrete classes called `OnState`, `OffState`, and `MuteState` implement this interface. The basic functionality can be described as follows. Initially, the TV is in the `Off` state. So, when you press the `On` button on the control panel, the TV moves to the `On` state. Then if you press the `Mute` button, it will go into the `Mute` state.

Assume that if you press the `Off` button when the TV is already in the `Off` state, if you press the `On` button when the TV is already in the `On` state, or if you press the `Mute` button when the TV is already in `Mute` mode, there will be no state change for the TV. The TV can go into the `Off` state from the `On` state or the `Mute` state (when you press the `Off` button). To help you understand this easily, Figure 22-1 describes this situation. (It is from my other book, *Design Patterns in C#* (Second Edition) which is published by the same publisher).

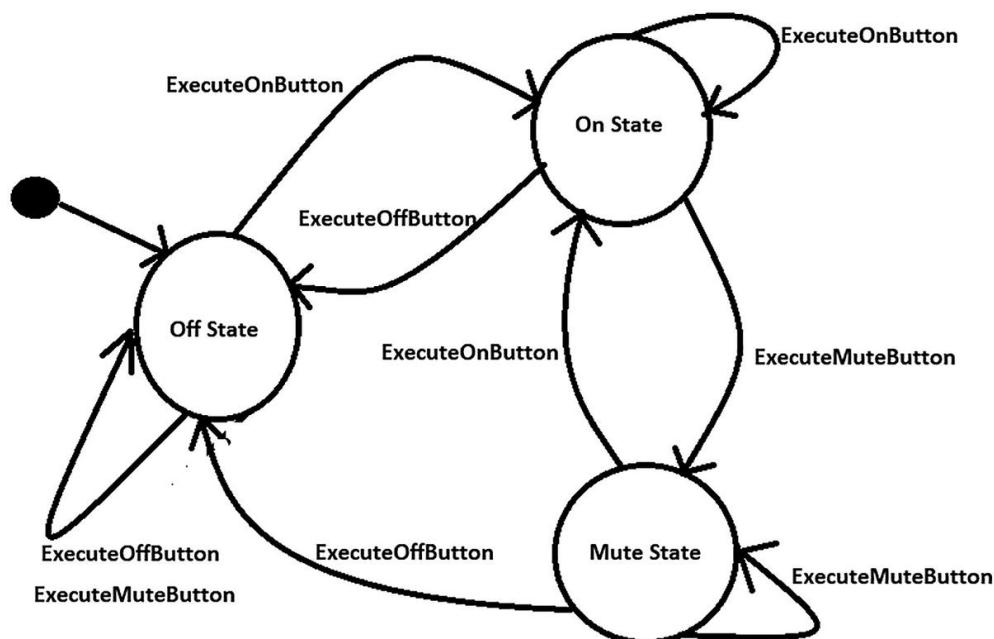


Figure 22-1 Different states of a TV

Points To Remember

- In this diagram, I have not marked any state as the final state, although in this illustration, in the end, I switch off the TV.
- To make the design simple, assume that if you press the `Off` (or `Mute`) button when the TV is already in the `Off` state or if you press the `On` button when the TV is already in the `On` state, or if you press the `Mute` button when the TV is already in the `Mute` mode, there will be no state change for the TV. But in the real world, a control panel may work differently. For example, if the TV is currently in the `On` state and you press the `Mute` button, the TV can go into `Mute` mode, and then if you press the `Mute` button again, the TV may come back to the `On` state. So, you may need to update your program logic accordingly.

- In Figure [22-1](#), the method names start with capital letters. But in the upcoming demonstration, following Java’s method naming convention, I make the first letter of each method lowercase, such as `executeOffButton()` and `executeOnButton`.

There is a class called `TV` in this example. I told you that the TV has a control panel to support on, off, and mute operations. So, inside the `TV` class, you see three different methods, which are defined as follows:

```
class Television {
    PossibleState currentState;
    // Some code skipped here
    public void executeOffButton() {
        System.out.println("You pressed the Off button.");
        // Delegating the state behavior
        currentState.pressOffButton(this);
    }
    public void executeOnButton() {
        System.out.println("You pressed the On button.");
        // Delegating the state behavior
        currentState.pressOnButton(this);
    }
    public void executeMuteButton() {
        System.out.println("You pressed the Mute button.");
        // Delegating the state behavior
        currentState.pressMuteButton(this);
    }
}
```

Notice that these methods invoke or “delegate to” the current state object’s method. For example, when you press `executeMuteButton()`, the control will invoke `pressMuteButton(...)` based on the current state of the television.

When you assign a new value to the `currentState` variable, the program appears to change its class because different code will be invoked from the same method call based on whichever `PossibleState` object is current.

Let’s follow the class diagram now.

Class Diagram

Figure [22-2](#) shows the key participants in the upcoming example.

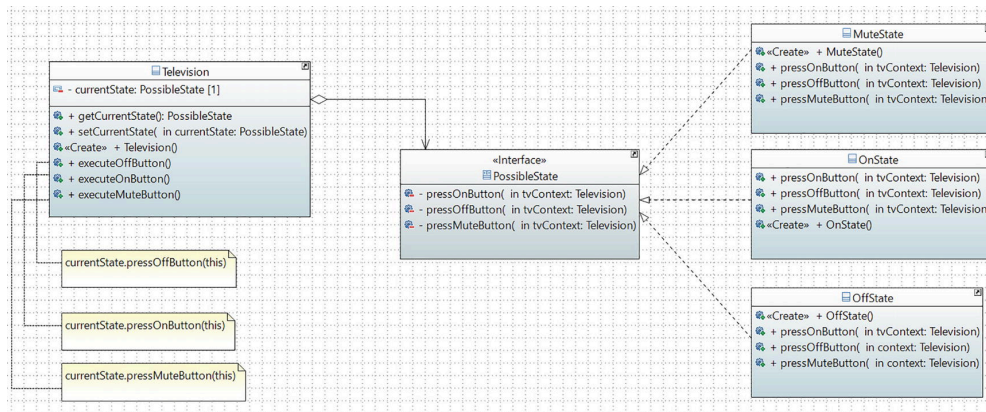


Figure 22-2 Class diagram

Package Explorer View

Figure [22-3](#) shows the high-level structure of the program. To make the diagram short and concise, I just expand one of the three possible states.

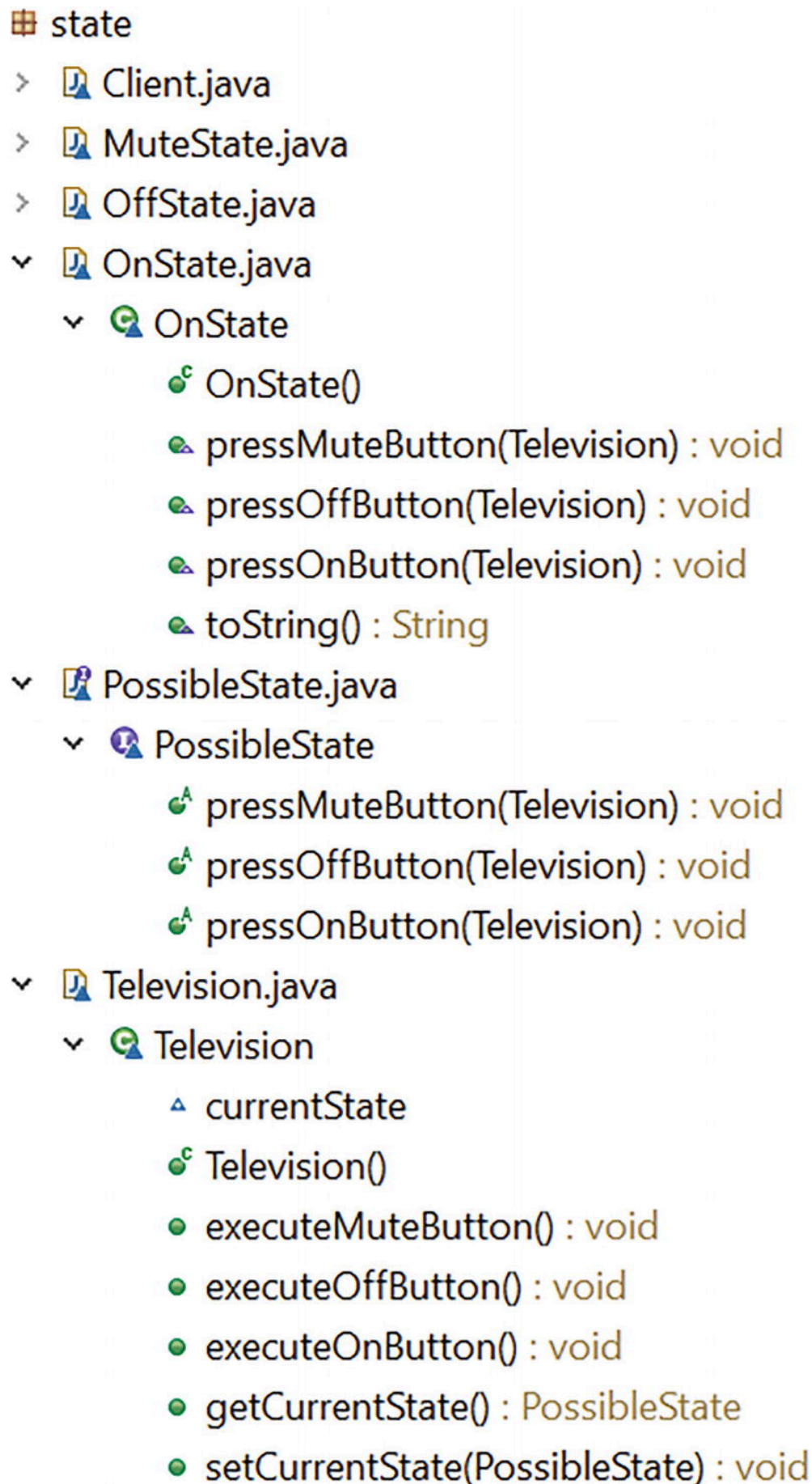


Figure 22-3 Package Explorer view

Demonstration

All parts of the program are stored inside the package `jdp3e.state`. Here's the complete implementation:

```
// PossibleState.java
interface PossibleState {
    // Users can press any of these buttons-On, Off or Mute
    void pressOnButton(Television tvContext);
    void pressOffButton(Television tvContext);
    void pressMuteButton(Television tvContext);
}

// OffState.java
// Subclasses do not contain any local state.
// Only one unique instance of IPossibleStates is required.
// This class describes the off state behavior
class OffState implements PossibleState {
    public OffState() {
        System.out.println("The TV is Off now.\n");
    }
    // TV was off. The user presses the On button now.
    @Override
    public void pressOnButton(Television tvContext) {
        System.out.println("The TV was Off. Going from Off to On state.");
        tvContext.setCurrentState(new OnState());
    }
    // TV was off. The user presses the Off button now.
    @Override
    public void pressOffButton(Television tvContext) {
        System.out.print("The TV was already in Off state.");
        System.out.println(" So, ignoring this operation.");
    }
    // TV was off. The user presses the mute button now.
    @Override
    public void pressMuteButton(Television tvContext) {
        System.out.print("The TV was already off.");
        System.out.println(" So, ignoring this operation.");
    }
    @Override
    public String toString() {
        return " Switched off.";
    }
}

// OnState.java
// This class describes the on state behavior

class OnState implements PossibleState {
    public OnState() {
```



```

        System.out.println("The TV is On now.\n");
    }
    // TV is already On. The user presses the On button now.
    @Override
    public void pressOnButton(Television tvContext) {
        System.out.print("The TV was already on.");
        System.out.println(" Ignoring repeated on button press operatio
    }
    // TV is already On. The user presses the off button now.
    @Override
    public void pressOffButton(Television tvContext) {
        System.out.println("The TV was on. So,switching off the TV.");
        tvContext.setCurrentState(new OffState());
    }
    // TV is already On. The user presses the mute button now.
    @Override
    public void pressMuteButton(Television tvContext) {
        System.out.println("The TV was on. So,moving to the silent mode
        tvContext.setCurrentState(new MuteState());
    }
    @Override
    public String toString(){
        return " Switched on.";
    }
}

// MuteState.java
class MuteState implements PossibleState {
    public MuteState() {
        System.out.println("The TV is in Mute mode now.\n");
    }
    // TV is muted, but the user presses the Off button
    @Override
    public void pressOnButton(Television tvContext) {
        System.out.print("The TV was in mute mode.");
        System.out.println(" So, moving to the normal state.");
        tvContext.setCurrentState(new OnState());
    }
    // TV is muted, but the user presses the Off button
    @Override
    public void pressOffButton(Television tvContext) {
        System.out.print(" The TV was in mute mode.");
        System.out.println(" So, switching off the TV.");
        tvContext.setCurrentState(new OffState());
    }
    // TV is muted, but the user press the mute button again
    @Override
    public void pressMuteButton(Television tvContext) {
        System.out.print("The TV was already in Mute mode.");
        System.out.println("So, ignoring this operation.");
    }
}

```



```

    }
    @Override
    public String toString(){
        return "Mute mode.";
    }
}
// Television.java
// Television is the context class
class Television {
    PossibleState currentState;
    public PossibleState getCurrentState() {
        return currentState;
    }
    public void setCurrentState(PossibleState currentState) {
        this.currentState = currentState;
    }
    public Television() {
        // Starting with Off state
        this.currentState = new OffState();
    }
    public void executeOffButton() {
        System.out.println("You pressed the Off button.");
        // Delegating the state behavior
        currentState.pressOffButton(this);
    }
    public void executeOnButton() {
        System.out.println("You pressed the On button.");
        // Delegating the state behavior
        currentState.pressOnButton(this);
    }
    public void executeMuteButton() {
        System.out.println("You pressed the Mute button.");
        // Delegating the state behavior
        currentState.pressMuteButton(this);
    }
}
// Client. Java
package jdp3e.state

;
class Client {
    public static void main(String[] args) {
        System.out.println("***State Pattern Demonstration.***\n");
        // TV is initialized with Off state.
        Television tv = new Television ();
        System.out.println("User is pressing buttons in the
                            following sequence:");
    }
}

```

```

System.out.println("Off->Mute->On->On->Mute->Mute->On->Off\n");
// TV is off. Pressing the off button again.
tv.executeOffButton();
// Pressing the mute button when the TV is off
tv.executeMuteButton();
//Checking the current state
System.out.println("Current state:"+ tv.getCurrentState());
// Switching on the television.
tv.executeOnButton();
//Checking the current state
System.out.println("Current state:"+ tv.getCurrentState());
// Pressing the On button again when the TV is On
tv.executeOnButton();
// Putting the TV in the mute mode.
tv.executeMuteButton();
// TV is already muted. Pressing Mute button again
tv.executeMuteButton();
// Checking the current state
System.out.println("Current state:"+ tv.getCurrentState());
// TV is silent now. Pressing the On Button now.
tv.executeOnButton();
// Switching off the TV now.
tv.executeOffButton();
System.out.println("Current state:"+ tv.getCurrentState());
    }
}

```

Output

Notice the state changes in the following output. I marked them in bold.

```

***State Pattern Demonstration***
The TV is Off now.
User is pressing buttons in the following sequence:
Off->Mute->On->On->Mute->Mute->On->Off
You pressed the Off button.
The TV was already in Off state. So, ignoring this operation.
You pressed the Mute button.
The TV was already off. So, ignoring this operation.
Current state: Switched off.
You pressed the On button.
The TV was Off. Going from Off to On state.
The TV is On now.
Current state: Switched on.
You pressed the On button.
The TV was already on. Ignoring repeated on button press operation.
You pressed the Mute button.

```

```
The TV was on. So, moving to the silent mode.  
The TV is in Mute mode now.  
You pressed the Mute button.  
The TV was already in Mute mode. So, ignoring this operation.  
Current state: Mute mode.  
You pressed the On button.  
The TV was in mute mode. So, moving to the normal state.  
The TV is On now.  
You pressed the Off button.  
The TV was on. So, switching off the TV.  
The TV is Off now.  
Current state: Switched off.
```

Analysis

The GoF mentions four important points about this pattern. Let's look at them:

- `Context` delegates state-specific requests to the current `ConcreteState` object.
- A context may pass itself as an argument to the `State` object handling the request. This lets the `State` object access the context if necessary.
- `Context` is the primary interface for clients. Clients can configure a context with `State` objects. Once a context is configured, its clients don't have to deal with the `State` objects directly.
- Either the `Context` or the `ConcreteState` subclass can decide which state succeeds another and under what circumstances.

Now notice your context class `Television`, any subset of `PossibleStates` such as `OffState`, and the client code. You can see that the example shown in the demonstration meets these criteria, such as

- You also delegate the task.
- You also use the context as an argument.
- The client interacts with the television objects only.
- When the client initializes the `Television` object, a state object is also initialized inside the `Television`'s constructor.
- The context object is used to set different states under different circumstances.

Q&A Session

22.1 Can you elaborate on how this pattern works in a real-world scenario?

Psychologists have repeatedly documented the fact that human beings perform their best when they are in a relaxed mood but in the reverse scenario, when their minds are filled with tension, they cannot produce great results. That is why they always suggest you work in a relaxed mood. So, the same work can be enjoyable or boring depending on your current mood.

Think about the demonstration example again. Suppose you want to watch the live telecast of the winning moments of your favorite team. To watch and enjoy the moment, you need to power on the TV first. If the TV is not functioning properly at that moment and cannot be in the On state, you cannot enjoy the moment. So, if you want to enjoy the moment through your TV, the first criterion is that the TV should change its state from Off to On. The State pattern is helpful if you want to design a similar kind of behavior change in an object when its internal state changes.

22.2 In this example, you considered only three states of a TV: On, Off, and Mute. There can be many other states; for example, there may be a state that deals with connection issues or different display conditions. Why have you ignored those issues?

The straightforward answer I ignored those states in order to keep the example simple. If the number of states increases significantly in the system, then it becomes difficult to maintain the system (and this is one of the key challenges associated with this design pattern). But if you understand this implementation, you can easily add any states you want.

22.3 I noticed that the GoF represent a similar structure for both the State pattern and the Strategy pattern in their famous book. I am confused by that.

Yes, the structures are similar, but you need to remember that their intents are different. The GoF clearly states that a strategy object encapsulates an algorithm, but a state object encapsulates a state-specific behavior. Let's analyze further.

When you use the Strategy pattern, you get a better alternative to subclassing. In the State design pattern, different types of behaviors can be encapsulated in a state object, and the context is delegated to any of these states. When a context's internal state changes, its behavior also changes. So, the State pattern can be thought of as a dynamic version of the Strategy pattern.

Consider a true case study. Once I lost my debit card on my way home. So, I needed to block the card immediately. In a situation like this, my bank provides various facilities, such as if you are a priority customer, you can block the card using your personal phone-banking pin. But for a normal customer, you need to connect with the customer care department and raise a request to block the card. Notice the end goal: I needed to block the card. So, I could opt for any of the available options to fulfill my need. The Strategy pattern helps you in a similar case because you can develop different algorithms to fulfill needs. Now think deeply: prior to using these facilities you must have a network connection, otherwise they are of no use. In other words, a customer can use the facility from a place where the network connection is active. This situation mimics the need for the State design pattern. So, it is quite possible that you need to use both patterns in an application.

You may note another interesting point: in the case of the Strategy pattern, the different algorithms are not dependent on each other. But, in the case of a State design pattern, one state can trigger a state transition to reach a different state. In such a case, one state may need to know about the other state.

Lastly, the State patterns can also help you avoid lots of `if` conditions in particular contexts. For example, consider the example once again. If the TV is in the Off state, it cannot go into the Mute state. From this state, it can move to the On state only. So, if you do not like the State design pattern, you may need to write code like this:

```
class TV {  
    //Some code before  
    public void executeOnButton(){  
        if(currentState==Off ){  
            System.out.println("You pressed On button. Going from Off to OnState");  
            //Some code after  
        }  
        if(currentState==On ){  
            System.out.println("You pressed On button. TV is already  
                               in On state. So, ignoring this operation.");  
            //Some code after  
        }  
        else {  
            System.out.println("TV was in mute mode.So, moving to the normal stat  
        }  
        public void executeOffButton() {  
            //Repeat the state check and proceed accordingly  
        }  
        public void executeMuteButton(){
```

```
// Repeat the state check and proceed accordingly
}
//Some code after if any
```

Notice that you need to repeat these checks for different kinds of button presses (for example, for the `executeOffButton()` and `executeMuteButton()` methods, you need to repeat these checks). So, if you do not think in terms of states, over a period of time, handling different conditions with lots of `if-else` statements is very challenging and it can be difficult when the codebase continually grows.

22.4 I understand that the control panel of the television supports three operations: on, off, and mute. But when I add a new state, I do not need to change this class. Is this correct?

Yes. Notice that each of these television states is closed for modification, but you can add a new state to this application. When you add a new state, you do not need to change the number of buttons on the control panel. Instead, you need to focus on the transition of states. This is why a context class (`Television` in this example) does not implement a state-specific behavior directly. It simply delegates the state-specific behavior to the correct class (`states` in this example).

22.5 What are the common characteristics between the Strategy pattern and the State pattern?

The State pattern can be seen as a dynamic Strategy pattern. Both patterns promote encapsulation and delegation. But as said before, a strategy object encapsulates an algorithm, but a state object encapsulates a state-specific behavior.

22.6 It appears to me that these state objects are acting like singletons. Is this correct?

Yes, it's a nice observation. The concrete subclasses of `PossibleStates` do not contain any local state in this example and as a result, in this application, only one state instance is working. Most of the time, this pattern acts in a similar way.

22.7 Why are you using the context as a method parameter? Can you avoid them in statements like the following:

```
void pressOnButton(Television tvContext);
```

Using the context, I'm using/saving states. Also, notice that the concrete subclasses of `PossibleState` do not contain any local state. So, in this application, only one state instance is working. This construct helps you to evaluate whether you need to change a current state.

You may see many applications that don't do the same. They may initialize the value inside the constructor of a state.

22.8 What are the pros and cons of the State design pattern?

The advantages are as follows:

- You have already seen that you can add new states and extend a state's behavior easily. Also, state behavior can be extended without any hassle. Notice that in this example, you can add a new state and new behavior for a `Television` class without changing this class.
- It reduces the use of `if-else` statements. In other words, the conditional complexity is reduced. (Refer to the answer to question 22.3 earlier.)

Here is a downside of using this pattern:

- The State pattern is also known as "Objects for States." So, you can assume that more states need more code and the obvious side effect is difficult maintenance for you.
- More states mean more classes for you. It tells you to put more effort into testing the code.
- The state transitions are not always straightforward. If there are various possibilities, overall implementation can be complex, too.
- It is not always easy to predict all possible states at the beginning.

22.9 In these implementations, `Television` is a concrete class. Why are you not programming to interface in this case?

I assume that the `Television` class is not going to change. So, I ignore that part to reduce the code size of the program. But yes, you can always start from an interface, in which you can define the necessary contracts.

22.10 In your example, I see that inside the client code when you execute the On, Off, or Mute buttons on the television's control panel, the states of the television change. Does it mean that a client needs to trigger these state transitions?

A state transition can be triggered in various ways. For example, you can use a method parameter to change a state, or you can use the context class to perform the same. Consider a banking transaction in this context: after a withdrawal, if you do not maintain the minimum balance, some of the features can be disabled, or you need to pay some penalty, which is not the case if you maintain a minimum balance. So, it is important to note that the “withdrawal amount” in the withdrawal method decides the next state of the account. So, a client can pass this value inside a withdrawal method.

In this example, when I initialize the `Television` class, I trigger the Off state, so I use the context class to trigger a state transition. This state changes when you press the On button.

22.11 You are initializing a `Television` object with an Off state. So can I say that both states and the context class can trigger the state transitions?

Yes.

22.12 Does the `Thread` class in Java follow the State design pattern?

I do not think so. The `Thread` class has several states such as NEW, RUNNABLE, BLOCKED, etc. These states are defined inside an `enum`. I do not see any substituting class based on these states so far.

Previous chapter

[< 21. Strategy Pattern](#)

Next chapter

[23. Mediator Pattern >](#)